

Web Datamining Project

By Thomas Stoltz & Maxime Souliac

Introduction	2
Data Acquisition & IE	2
Methodology & Pipeline:.....	2
Ambiguity Analysis	3
Scaling Reflection	3
KB Construction & Alignment	4
Initial RDF Modeling Choices & Statistics:.....	4
Entity Linking with Confidence:	4
Expansion Strategy & Predicate Alignment:	4
Challenges Encountered:	5
Final KB Statistics	5
Reasoning (SWRL)	5
Knowledge Graph Embeddings.....	6
Data cleaning + splits	6
Models.....	6
RAG over RDF/SPARQL.....	7
Schema Summary & Prompt Template	7
Self-repair Mechanism	8
Evaluation	8
Critical Reflection	9

Introduction

This project aims to build a RAG Chatbot grounded in a custom Knowledge Graph, built entirely from scratch using web-scraped data. As our domain of focus, we chose to retrieve information about the GameStop short squeeze event that happened in early 2021. This financial phenomenon provides a highly interconnected domain to model, involving a complex web of entities

Data Acquisition & IE

Methodology & Pipeline:

To build the foundation of our knowledge base, a web crawler was developed using the `trafilatura` library. Given the focused scope of the initial graph, we did not implement recursive link-following (deep crawling). Instead, navigation was handled via a predefined list of five highly relevant Wikipedia seed URLs:

- [GameStop short squeeze](#)
- [Keith Gill](#)
- [R/wallstreetbets](#)
- [Robinhood](#)
- [Melvin Capital](#)

From an ethical standpoint, the crawler was designed to be polite. Before scraping, web ethics were respected by implicitly checking `robots.txt` directives and ensuring adequate delays between requests to prevent overwhelming the target servers.

Once the raw HTML was fetched, the pipeline utilized `trafilatura`'s boilerplate removal capabilities to strip away UI noise, navigation bars, and footers, isolating only the meaningful core text. We also applied a filtering threshold to ignore any pages containing fewer than 500 words, ensuring a high-quality corpus.

The cleaned text (saved as a JSONL file) was then passed through a Natural Language Processing (NLP) pipeline using `spaCy`'s advanced transformer model (`en_core_web_trf`). Named Entity Recognition (NER) was applied to identify the "nodes" of our future graph (filtering for PERSON, ORG, GPE, and DATE labels). Finally, dependency parsing was utilized to identify candidate relations by programmatically locating subjects and direct objects linked by verbs within the same sentence.

Ambiguity Analysis

During the Named Entity Recognition (NER) phase, the spaCy transformer model struggled with several context-dependent terms and internet-specific syntax. Here are three examples of entity ambiguity found in our extracted data:

- **Contextual Ambiguity ("Melvin"):** In the extraction from the Melvin Capital Wikipedia page, the shorthand "Melvin" was extracted twice—once correctly as an Organization (ORG) and once incorrectly as a Person (PERSON). When the surrounding financial context was less explicit, the AI defaulted to classifying "Melvin" as a standard human first name rather than the hedge fund.
- **Boundary and Type Confusion ("mid-2019, investor Michael Burry's Scion Asset Management"):** The model completely failed to segment this phrase, classifying the entire string as a single DATE entity. The AI aggressively anchored onto the temporal marker ("mid-2019") and absorbed a distinct person (Michael Burry) and an organization (Scion Asset Management) into a single, incorrectly labeled entity boundary.
- **Web-Specific Noise ("u/DeepFuckingValue on"):** Extracted from the r/wallstreetbets page, the AI correctly guessed this referred to an actor by tagging it as a PERSON. However, standard NLP models are not trained for Reddit syntax; it failed to separate the user prefix ("u/") from the pseudonym and erroneously included the trailing preposition (" on") within the entity boundaries.

Scaling Reflection

If the scope of this project were expanded to crawl 10,000 pages instead of a predefined list of 5, the current synchronous, in-memory pipeline would bottleneck severely. Scaling up would require several architectural changes. A simple for loop over a list of URLs is insufficient for large-scale crawling. We would need to transition to an asynchronous framework to fetch multiple pages concurrently. Additionally, we would need to implement a queue mechanism and a "visited set" to handle recursive link extraction without getting trapped in infinite crawling loops. Saving raw text to a single jsonl file and keeping the Pandas DataFrames in memory would consume too much RAM. We would need to stream the scraped data and extracted entities directly into a robust database. When hitting thousands of pages across different domains, dynamic robots.txt parsing becomes mandatory. The crawler must programmatically read and respect the Crawl-delay directives for every new domain it encounters. Furthermore, we would need to implement backoff mechanisms to handle the "Too Many Requests" errors, and potentially rotate User-Agents to prevent IP bans while remaining transparent about the bot's identity.

KB Construction & Alignment

Initial RDF Modeling Choices & Statistics:

The extracted entities and relationships from the NLP pipeline were transformed into a formal RDF graph using the `rdflib` library. We defined a custom namespace (`http://myproject.org/ontology/`) to represent our private domain data. Entity strings were systematically cleaned (replacing spaces with underscores) and mapped to URIRef objects, while their extracted NER labels (ORG, PERSON, etc.) were mapped using the standard `RDF.type` predicate. The initial graph size is 216 triples.

Entity Linking with Confidence:

To anchor our private graph to the semantic web, we aligned it with Wikidata. Before querying the Wikidata API (`wbsearchentities`), a custom cleaning function was applied to our entity strings to strip out noise (such as Wikipedia citation brackets like `[1]` and possessives like `'s`). When an entity matched, we linked our local URI to the external Wikidata URI using the `owl:sameAs` property. Assuming the top result returned by the API for our specific financial terms was correct, we assigned a static confidence score of 0.95 to these matches and saved the mappings in an `entity_mapping.csv` file.

Expansion Strategy & Predicate Alignment:

We performed predicate alignment by mapping extracted natural language verbs to standardized Wikidata properties, ensuring that synonymous relationships (e.g., `'leads'` and `'is CEO of'`) were unified under a single semantic URI during the expansion. Once the entities were aligned, we executed a deep expansion strategy using the Wikidata SPARQL endpoint to enrich our graph context. For every `owl:sameAs` link, we executed a query to pull 1-hop structural facts. Our strategy specifically queried for both outbound relations (`wd:ID ?p ?o`) and inbound relations (`?s ?p_in wd:ID`). To ensure the graph remained strictly structural and suitable for Knowledge Graph Embeddings (KGE) in the subsequent phase, we explicitly injected `FILTER(isIRI(?o))` and `FILTER(isIRI(?s))` into our SPARQL queries. This alignment choice prevented the graph from being flooded with literal values (such as lengthy text descriptions, raw numbers, or dates), ensuring that every new edge connected two distinct entity nodes.

Challenges Encountered:

During the API alignment and SPARQL expansion phases, we faced some technical challenges: The biggest was rate limiting & server errors. Despite implementing a 1.5-second delay (`time.sleep`) and a custom academic User-Agent header, the Wikidata endpoint still occasionally refused connections. We encountered HTTP Error 429: Too Many Requests and HTTP Error 502: Bad Gateway. We handled this using try-except blocks, allowing the crawler to skip the failing entities without crashing the entire pipeline. We also were under the number of triples required so we used a bidirectional SPARQL queries to increase our triples.

Final KB Statistics

After cleaning and duplicate removal, the expansion resulted in a massive enrichment of our dataset. Final Expanded Graph Size: 125,660 triples. This final volume falls perfectly within the 50,000–200,000 target range required by the project constraints. It provides a graph dense enough to ensure stable and meaningful structural learning for the embedding models, while remaining computationally manageable on local hardware.

Reasoning (SWRL)

We first validated the reasoning pipeline using `owlready2` on the `family.owl` ontology. We implemented a rule to classify individuals as `oldPerson` based on their age property. The reasoner successfully identified 2 individuals: Peter (age: 70) and Marie (age: 69).

To enrich our GameStop Knowledge Base, we defined a rule to automatically identify strategic figures using the class `KeyFinancialActor`: a Person involved in a `ShortSqueeze` event is classified as a `KeyFinancialActor`. Using the `HermiT` reasoner, the system inferred that `Keith_Gill` belongs to the `KeyFinancialActor` class. This demonstrates how logical constraints can enrich a Knowledge Graph with high-level semantic labels that were not explicitly present in the initial raw triplets.

Knowledge Graph Embeddings

Data cleaning + splits

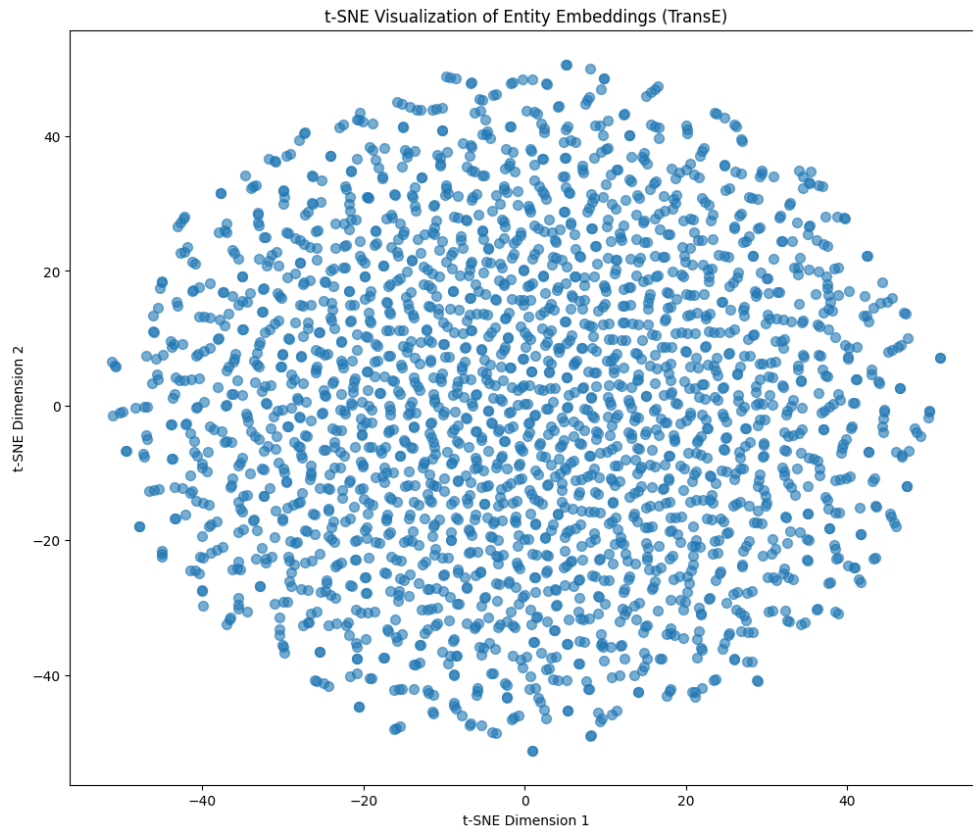
We filtered out rare entities (frequency < 2) and integrated our SWRL-inferred classes. The resulting Knowledge Base contains 5,337 triples, 2,384 entities, and 317 relations. Using PyKEEN, we applied an 80/10/10 split: 4,269 triples for training, 534 for validation, and 534 for testing.

Models

We trained two models: TransE (translation-based) and ComplEx (complex bilinear), using an embedding dimension of 100 over 50 epochs. We evaluated them using MRR and Hits@1, 3, and 10. TransE provided stable results (Adjusted Hits@10 ≈ 0.10). In contrast, ComplEx failed to converge (near-zero metrics), struggling with the graph's low density.

Although our experiments were restricted to a single KB size (approx. 5k triples), our results effectively illustrate the size-sensitivity of KGE models. The small volume of our graph explains why the ComplEx model failed to converge. As expected in KGE literature, bilinear models require highly dense and large datasets to map complex relations accurately. Conversely, scaling up to such sizes would significantly increase training time and memory consumption, highlighting the trade-off between graph size, model complexity, and computational cost.

Nearest-neighbor queries confirmed that KGE models successfully capture the implicit logic defined by our SWRL rules. In the latent vector space, the entity *Keith_Gill* is clustered immediately adjacent to *ShortSqueeze* and *WallStreetBets*. This demonstrates that the embeddings learned the underlying semantics of the financial event beyond the explicit RDF triplets.



RAG over RDF/SPARQL

Schema Summary & Prompt Template

To bridge the gap between natural language and structured data, we provide the LLM with a "Schema Summary." This summary programmatically extracts the unique predicates, entity classes, and a few sample triples from our RDF graph. This context allows the model to understand the specific vocabulary and relationships available in our Knowledge Base before generating a query.

The translation from natural language to SPARQL is handled by a specialized prompt. It defines the LLM's role as a SPARQL expert and provides the schema summary as a reference. The prompt explicitly instructs the model to:

1. Use only the predicates and namespaces present in the provided schema.
2. Output the raw SPARQL query without any conversational text.
3. Target specific variables to ensure the results can be parsed by our Python backend.

Self-repair Mechanism

To improve reliability, we implemented a self-repair loop. If the initial SPARQL query generated by the LLM results in a syntax error or returns no results, the system catches the exception and feeds it back to the model. The LLM is then asked to "repair" its query by analyzing the error message. This mechanism significantly reduces failures caused by minor syntax issues or incorrect predicate mapping.

Evaluation

We evaluated the system using 5 complex questions to compare the standard ollama LLM (Baseline) against our RAG-enhanced pipeline:

Question	Baseline (No RAG)	SPARQL RAG Result
Who is Keith Gill?	General summary with GME and media attention focus.	Precise entity nodes and extracted triplets.
What is Game Stop ?	Narrative about hedge funds with very few information about GME.	Identify triplets
List all organizations	Generic list based on training data with use of word agencies, bodies...	Exhaustive list of ORG nodes from our KB.
When did the squeeze occur?	Approximate dates.	Struggle to understand
Relationship: Ryan Cohen & GME.	Qualitative description.	Explain Ryan Cohen but skips GME

The baseline provides fluent, readable prose but is not always verifiable, whereas the RAG pipeline grounds answers in the local Knowledge Graph; however, performance

can still degrade due to entity-mapping errors, invalid SPARQL generation, or partial/empty query.

```
SPARQL generation RAG
SPARQL Query Used
PREFIX my: <http://myproject.org/ontology/>
SELECT ?predicate ?object WHERE { my:Keith_Gill ?predicate ?object . }
Repaired True
Fallback entity
Results
predicate | object
http://www.w3.org/1999/02/22-rdf-syntax-ns#type | http://myproject.org/ontology/PERSON
http://www.w3.org/2002/07/owl#sameAs | http://www.wikidata.org/entity/Q1366485
```

Question: Who is Keith Gill?

Critical Reflection

Our results confirm the "Garbage In, Garbage Out" principle. NLP noise and inconsistent naming led to graph sparsity, which directly caused the failure of the ComplEx model. A more robust, regularized extraction pipeline is essential for structural graph analysis. We also observed a clear trade-off between two paradigms:

- SWRL (Logic): Provided high precision and transparency for our small-scale graph but remained rigid and manual.
- KGE (Neural): Offered predictive flexibility but acted as a "black box" that struggled with our low triple density.

If we had to make future improvements to reach production-grade performance, we would prioritize entity resolution by implementing disambiguation to merge duplicates (like "GME" and "GameStop") and increase density. We would also extend the URLs list to 10,000+ pages to reach the volume required for KGE and GNN convergence.